

AIVANCITY, La Grande Ecole De L'intelligence Artificielle Et De La Data



Paris Île-de-France · Nice Côte d'Azur

COURSE: NO SQL REPORT (Technical Project Report)

PROJECT TITLE: CAMPUS EVENT MANAGER

Date: 12th September 2026

Submitted By

Kavya BASAPPA

Under the Guidance of

Prof. Etienne MAUFFRET

Prof. Madjid TAOUALIT

1. Project Overview

Campus Event Manager is a web application developed as part of the NoSQL project. The objective is to design and implement a MongoDB database and a lightweight web application for managing campus users, events, registrations and analytical information.

The application uses MongoDB as its main database and Flask with PyMongo for the web application layer. The project demonstrates NoSQL data modeling, embedded documents, document references, CRUD operations, validation, indexing and MongoDB aggregation pipelines.

The system manages two main collections: users and events. Users contain information such as name, email, department, role and interests. Events contain information about the event, its location, organizer, capacity, tags and embedded registrations.

The application provides five main functional areas:

- Dashboard
- Events
- Event Details & Registrations
- Users
- Analytics

The database is initialized and populated through reproducible MongoDB scripts stored in the database/ directory.

2. Technologies Used

Technology	Purpose
Python 3.14	Application development
Flask	Web framework
PyMongo	MongoDB connectivity
MongoDB	NoSQL database
MongoDB Shell	Database initialization and seeding
Jinja2	HTML templating
HTML/CSS	User interface
python-dotenv	Environment configuration
Git/GitHub	Version control

Campus Event Manager

Dashboard Events Users Analytics

Dashboard

Overview of campus events and registrations.

Total Users

15

Total Events

18

Upcoming Events

12

Total Registrations

69

Next 5 Upcoming Events

Career

Career Networking Evening

Start: 20/09/2026 18:00

Location: Business Center, D201, Paris

Confirmed: 5 / 50



Student Activity

Research Showcase

Start: 25/09/2026 14:00

Location: Research Building, R101, Paris

Confirmed: 0 / 45



Hackathon

Hackathon Kickoff

Start: 05/10/2026 09:00

Location: Innovation Center, B101, Paris

Confirmed: 6 / 60



Talk

Sustainability Forum

Start: 18/10/2026 14:00

Location: Main Hall, A201, Paris

Confirmed: 2 / 40



Meetup

Open Source Meetup

Start: 30/10/2026 18:00

Location: Technology Building, C102, Paris

Confirmed: 4 / 40



Most Popular Event

Sports

Campus Sports Tournament

Confirmed registrations: 12

Capacity: 12



Dashboard

3. NoSQL Data Model

The application uses two main MongoDB collections in the campus_events database:

```
campus_events
├── users
└── events
```

3.1 Users Collection

A user document contains:

```
_id
firstName
lastName
email
department
role
interests[]
createdAt
```

The interests field is represented as an array because one user can have multiple interests. A unique index is created on email to prevent duplicate users.

3.2 Events Collection

An event document contains:

```
_id
title
description
category
tags[]
startDate
endDate
capacity
location
organizerId
registrations[]
createdAt
```

The location is embedded directly inside the event:

```
location
├── building
├── room
└── campus
```

Registrations are also embedded:

```

registrations[]
├── userId
├── registeredAt
└── status

```

The event organizer is represented through organizerId, which references a document in the users collection.

3.3 Modeling Decisions

Embedding location: Location is embedded because building, room and campus belong directly to an event and are normally displayed together with the event. Embedding avoids the need for a separate location collection and additional queries.

Embedding registrations: Registrations are embedded within events because registration information is closely associated with a specific event. This also allows MongoDB array operations such as \$push and \$pull to add and remove registrations directly.

Referencing users: Users are stored separately because users exist independently from events and can participate in multiple events. The registration document therefore stores userId rather than duplicating the complete user information.

Referencing the organizer: organizerId references the corresponding user rather than embedding the entire organizer document.

```

(.venv) PS C:\Users\Kavya B\OneDrive\Documents\AIVANCITY\campus-event-manager> mongosh
Current Mongosh Log ID: 6aa5657d70db15ffb2fd73ee
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&ap
pName=mongosh+2.10.0
Using MongoDB:      8.3.8
Using Mongosh:      2.10.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

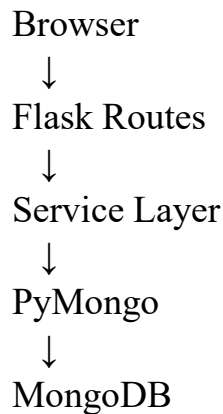
-----
  The server generated these startup warnings when booting
  2026-09-12T19:25:23.165+05:30: Access control is not enabled for the database. Read and write access t
o data and configuration is unrestricted
-----

test> use campus_events
switched to db campus_events
campus_events> show collections
events
users
campus_events> db.users.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { email: 1 },
    name: 'idx_users_email_unique',
    unique: true
  }
]
campus_events> db.events.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { startDate: 1 }, name: 'idx_events_startDate' },
  { v: 2, key: { category: 1 }, name: 'idx_events_category' }
]
campus_events>

```

4. Application Architecture

The application follows a simple layered Flask architecture.



4.1 Routes

The routes/ directory contains the Flask route modules:

- dashboard.py
- events.py
- users.py
- analytics.py

These modules receive HTTP requests and return the appropriate HTML pages.

4.2 Services

The services/ directory contains database and application logic:

- mongo_service.py
- dashboard_service.py
- event_service.py
- user_service.py
- analytics_service.py

Separating routes from services keeps database operations and business logic organized.

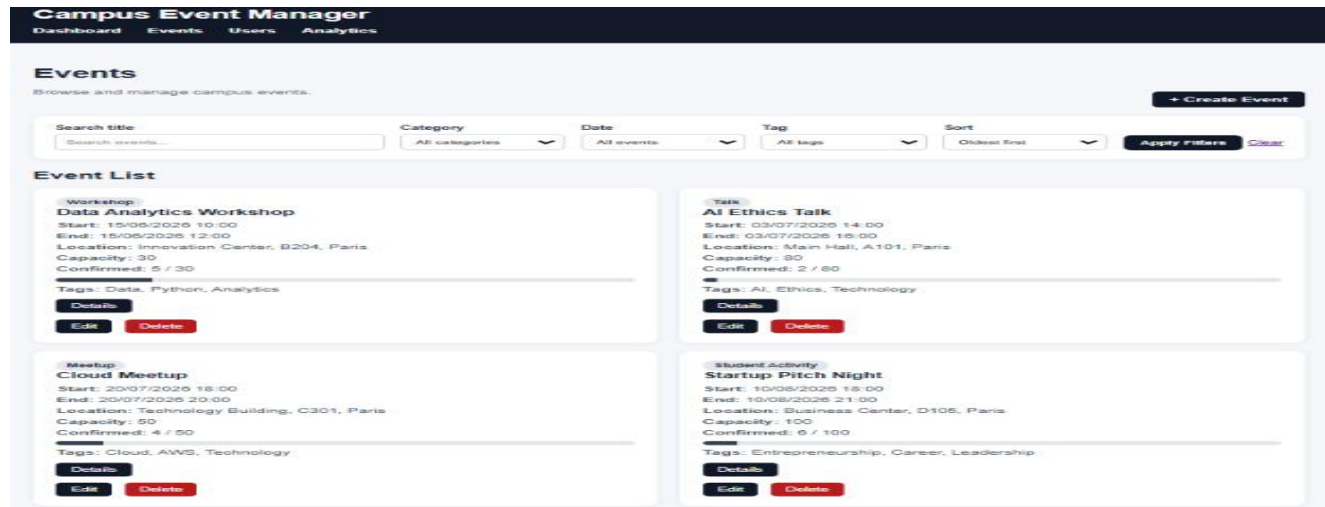
4.3 Templates

The templates/ directory contains the Jinja2 HTML templates for the different application pages.

5. Application Pages

5.1 Events Page

The Events page displays the complete event list and provides title search, category filtering, tag filtering, upcoming/past filtering, date sorting, capacity indicators, confirmed registration count, create, edit, delete and event details.



Events

5.2 Event Details & Registrations

The Event Details page displays the complete event information, including description, category, tags, dates, location, organizer, capacity and occupancy.

Users can be selected from existing users and registered for an event. The application prevents duplicate registrations and rejects confirmed registrations when the event has reached its capacity.

Registrations are updated directly in the embedded registrations array using MongoDB array operators such as \$push and \$pull.

Campus Event Manager
Dashboard Events Users Analytics

Create Event

Creates a new campus event.

Title *

Description *

Category * **Capacity ***

Tags
e.g. AI, Data, Python
Separate multiple tags with commas.

Start date & time * **End date & time ***

Location
Building * **Room *** **Campus ***

Organizer *
Select an organizer

Create Event Cancel

Campus Event Manager — MongoDB & Flask Project

5.3 Users Page

The Users page provides a searchable directory of campus users. It supports search by name or email, department filtering, role filtering, registration count, user creation, editing, deletion and user details.

The user details page displays the user's information, interests and registered events. It also separates upcoming and past registrations. Users with zero registrations are supported.

Campus Event Manager
Dashboard Events Users Analytics

Users

Manage campus users and their event participation.

+ Create User

Search **Department** **Role**

Name or email... All departments All roles Apply Filters Clear

User Directory + Create User

student
Lucas Bernard
Email: lucas.bernard@campus.fr
Department: Cloud Computing
Interests: Cloud, Technology, Open Source
Registrations: 7
Details Edit Delete

student
Sofia Dubois
Email: sofia.dubois@campus.fr
Department: Data Engineering
Interests: Data, Python, Analytics
Registrations: 5
Details Edit Delete

faculty
Gabriel Fournier
Email: gabriel.fournier@campus.fr
Department: Research
Interests: Research, Innovation, Technology
Registrations: 0
Details Edit Delete

student
Louis Garcia
Email: louis.garcia@campus.fr
Department: Computer Science
Interests: AI, Technology, Research
Registrations: 3
Details Edit Delete

Users

5.4 Analytics Page

The Analytics page contains six MongoDB aggregation analyses required by the project:

Analysis	Purpose
A. Registrations by Category	Events and confirmed registrations by category
B. Top Five Events	Ranks events by confirmed registrations and occupancy
C. Users with No Registration	Identifies users without event registrations
D. Above Average Occupancy	Finds events above average occupancy
E. Most Used Tags	Counts tag usage with array aggregation
F. Events by Month	Groups events and registrations by month

Campus Event Manager
Dashboard Events Users Analytics

Analytics

MongoDB aggregation analysis of campus events and registrations.

A. Registrations by Category

Category	Number of Events	Total Confirmed
Workshop	3	13
Meetup	3	13
Sports	1	12
Talk	4	10
Career	2	7
Hackathon	1	6
Student Activity	4	6

B. Top 5 Events

Event	Category	Capacity	Confirmed	Occupancy
Campus Sports Tournament	Sports	12	12	100.0%
Startup Pitch Night	Student Activity	100	6	6.0%
Hackathon Kickoff	Hackathon	60	6	10.0%
Student Leadership Meetup	Meetup	30	5	16.7%
Data Analytics Workshop	Workshop	30	5	16.7%

C. Users With No Registration

Name	Email	Department	Role
Gabriel Fournier	gabriel.fournier@campus.fr	Research	faculty
Paul Girard	paul.girard@campus.fr	Administration	staff
Alice Mercier	alice.mercier@campus.fr	Data Science	faculty

Analytics

5. MongoDB Aggregation Focus

6.1 Users with No Registration

The users-with-no-registration analysis uses \$lookup to connect users with event registrations. The pipeline compares the independent users collection with registration information stored inside event documents.

```
graph TD
    A[users] --> B["$lookup"]
    B --> C[match registration references]
    C --> D[identify users without matches]
    D --> E["$project"]
    E --> F["$sort"]
```

This analysis is useful for identifying users who have not yet participated in any campus event.

6.2 Events Above Average Occupancy

The occupancy analysis first calculates confirmed registrations for each event and derives an occupancy percentage:

$$\text{confirmed registrations} / \text{capacity} \times 100$$

The aggregation then calculates the average occupancy across events and compares each event against that average. The pipeline demonstrates multiple MongoDB aggregation stages, including \$filter, \$group, \$avg, \$addFields, \$match and \$project.

7. Validation and Indexing

MongoDB schema validation was implemented for both collections. Examples of validation rules include required fields, valid email format, unique email addresses, controlled user roles, non-empty event titles, capacity greater than zero, valid event dates, controlled registration statuses and correct embedded document structures.

The following indexes are created:

```
users.email  
events.startDate  
events.category
```

The email index is unique. These indexes support common operations such as finding users by email, sorting events by date and filtering events by category.

8. Difficulties and Limitations

One of the main challenges was designing a MongoDB model that correctly balanced embedded documents and references.

Another challenge was implementing registration rules while keeping registration information embedded within events. The application needed to prevent duplicate registrations and prevent confirmed registrations when an event was full.

The project currently uses a local MongoDB instance and a simple Flask interface. It does not include authentication, cloud deployment or advanced role-based access control.

9. Possible Improvements

- User authentication and role-based permissions
- Cloud MongoDB deployment
- Email notifications and event reminders
- Pagination for large datasets
- More advanced dashboards and charts
- REST API endpoints
- Automated testing
- Docker-based deployment

10. Conclusion

Campus Event Manager demonstrates how MongoDB can be used to build a practical NoSQL application. The project combines document modeling, embedded arrays, references, validation, indexes, CRUD operations and aggregation pipelines within a Flask application.

The resulting application provides complete management of users, events and registrations while also offering analytical views of event participation.

The project also demonstrates the advantages of MongoDB's flexible document model, particularly for embedded event locations and registration arrays, while maintaining references for reusable user information.